

Dense baseline (naive, seq=512, bs=32, fp32)

| Task   | Metric  | Dense score | Latency (ms) | Throughput (samples/s) | Peak mem infer (MB) |
|--------|---------|-------------|--------------|------------------------|---------------------|
| CoLA   | MCC     | 53.39       | 120.34       | 262.6                  | 987.2               |
| SST-2  | Acc     | 92.43       | 110.74       | 281.2                  | 987.2               |
| MRPC   | F1      | 91.35       | 112.40       | 279.2                  | 987.2               |
| QQP    | F1      | 87.82       | 127.00       | 251.9                  | 987.2               |
| MNLI-m | Acc     | 84.58       | 125.74       | 254.3                  | 987.2               |
| QNLI   | Acc     | 91.54       | 125.51       | 254.5                  | 987.2               |
| RTE    | Acc     | 72.56       | 111.82       | 275.2                  | 987.2               |
| STS-B  | Pearson | 88.05       | 119.64       | 266.7                  | 987.2               |
| G-AVG  | —       | 82.72       | 119.1        | 265.7                  | 987.2               |
| A-AVG  | Acc(4)  | 85.28       | —            | —                      | —                   |

注：为何 Dense 987 MB 而压缩 Naive 约 2004 MB？ HF BERT 在 PyTorch ≥ 2.0 下默认走 SDPA 融合路径，不物化 [B,H,M,M] logits 与 attention weights； 压缩 Naive(einsum) backend 显式计算这两个矩阵（各 ≈ 384 MB，合计 ≈ 768 MB 额外开销）。这是 kernel 实现差异，非对比不公平。FlashSVD (~708 MB) 才是与 Dense 同水位的公平对比。详见 Issues\_found.md #19。

压缩未微调 (naive backend, per-head ra48) ——精度对比 + Δ vs Dense

| Task            | Dense | SVD   | Δ      | FWSVD | Δ      | DRONE | Δ      | AdaSVD | Δ      |
|-----------------|-------|-------|--------|-------|--------|-------|--------|--------|--------|
| CoLA (MCC)      | 53.39 | 2.64  | -50.75 | 14.44 | -38.95 | 1.57  | -51.82 | 0.20   | -53.18 |
| SST-2 (Acc)     | 92.43 | 71.79 | -20.64 | 77.75 | -14.68 | 85.55 | -6.88  | 61.35  | -31.08 |
| MRPC (F1)       | 91.35 | 0.00  | -91.35 | 37.20 | -54.15 | 84.72 | -6.63  | 0.00   | -91.35 |
| QQP (F1)        | 87.82 | 17.17 | -70.65 | 68.00 | -19.82 | 76.22 | -11.60 | 51.54  | -36.28 |
| MNLI-m (Acc)    | 84.58 | 37.35 | -47.23 | 52.23 | -32.36 | 57.87 | -26.71 | 34.87  | -49.72 |
| QNLI (Acc)      | 91.54 | 54.64 | -36.90 | 57.33 | -34.21 | 60.21 | -31.34 | 45.84  | -45.71 |
| RTE (Acc)       | 72.56 | 47.29 | -25.27 | 58.48 | -14.08 | 59.57 | -13.00 | 53.43  | -19.13 |
| STS-B (Pearson) | 88.05 | 35.22 | -52.82 | 69.34 | -18.70 | 49.30 | -38.75 | 63.67  | -24.38 |
| G-AVG           | 82.72 | 33.26 | -49.45 | 54.35 | -28.37 | 59.38 | -23.34 | 38.86  | -43.85 |
| A-AVG           | 85.28 | 52.77 | -32.51 | 61.45 | -23.83 | 65.80 | -19.48 | 48.87  | -36.41 |

压缩后微调 (post-compress finetune, per-head ra48) ——精度对比 + Δ vs 对照基线

| Task | Base (Dense or Dense_finetuned) | SVD-ft | Δ | FWSVD-ft | Δ | DRONE-ft | Δ | AdaSVD-ft | Δ |
|------|---------------------------------|--------|---|----------|---|----------|---|-----------|---|
|------|---------------------------------|--------|---|----------|---|----------|---|-----------|---|

| Task               | Base (Dense or<br>Dense_finetuned) | SVD-<br>ft   | $\Delta$ | FWSVD-<br>ft | $\Delta$ | DRONE-<br>ft | $\Delta$ | AdaSVD-<br>ft | $\Delta$ |
|--------------------|------------------------------------|--------------|----------|--------------|----------|--------------|----------|---------------|----------|
| CoLA<br>(MCC)      | 57.83                              | 37.46        | -20.37   | 47.17        | -10.66   | 43.13        | -14.70   | 41.08         | -16.75   |
| SST-2<br>(Acc)     | 92.55                              | 91.06        | -1.49    | 90.48        | -2.06    | 90.83        | -1.72    | 91.63         | -0.92    |
| MRPC<br>(F1)       | 91.35                              | 84.33        | -7.02    | 88.55        | -2.80    | 90.19        | -1.16    | 88.47         | -2.87    |
| QQP (F1)           | 87.82                              | 87.28        | -0.54    | 87.39        | -0.43    | 87.26        | -0.56    | 87.41         | -0.41    |
| MNLI-m<br>(Acc)    | 84.58                              | 82.01        | -2.58    | 82.73        | -1.85    | 82.01        | -2.58    | 82.33         | -2.25    |
| QNLI<br>(Acc)      | 91.54                              | 88.91        | -2.64    | 89.11        | -2.43    | 89.33        | -2.21    | 89.18         | -2.36    |
| RTE (Acc)          | 72.56                              | 61.37        | -11.19   | 64.62        | -7.94    | 74.01        | +1.44    | 59.57         | -13.00   |
| STS-B<br>(Pearson) | 88.05                              | 86.59        | -1.46    | 87.00        | -1.05    | 84.87        | -3.18    | 87.29         | -0.75    |
| <b>G-AVG</b>       | <b>83.29</b>                       | <b>77.37</b> | -5.91    | <b>79.63</b> | -3.65    | <b>80.20</b> | -3.08    | <b>78.37</b>  | -4.91    |
| <b>A-AVG</b>       | <b>85.31</b>                       | <b>80.84</b> | -4.48    | <b>81.74</b> | -3.57    | <b>84.04</b> | -1.27    | <b>80.68</b>  | -4.63    |

System Performance (per-head ra48, seq=512, bs=32, fp32)

格式: Naive = lat(ms)/thr(sps)/mem(MB), Flash = 同格式

| Task  | Method | Naive        | Flash       | Speedup | Memory Change     |
|-------|--------|--------------|-------------|---------|-------------------|
| CoLA  | SVD    | 161/196/2004 | 92/348/708  | x1.75   | -1296 MB (-64.7%) |
| CoLA  | FWSVD  | 158/200/2011 | 95/338/708  | x1.67   | -1303 MB (-64.8%) |
| CoLA  | DRONE  | 163/194/2004 | 94/339/708  | x1.73   | -1296 MB (-64.7%) |
| CoLA  | AdaSVD | 166/190/2522 | 101/317/725 | x1.65   | -1797 MB (-71.3%) |
| SST-2 | SVD    | 160/195/2004 | 91/352/708  | x1.75   | -1296 MB (-64.7%) |
| SST-2 | FWSVD  | 157/198/2011 | 93/345/708  | x1.69   | -1303 MB (-64.8%) |
| SST-2 | DRONE  | 161/194/2004 | 93/345/708  | x1.73   | -1296 MB (-64.7%) |
| SST-2 | AdaSVD | 164/190/2516 | 99/322/722  | x1.65   | -1794 MB (-71.3%) |
| MRPC  | SVD    | 161/194/2004 | 94/342/708  | x1.72   | -1296 MB (-64.7%) |
| MRPC  | FWSVD  | 158/199/2011 | 93/344/708  | x1.70   | -1303 MB (-64.8%) |
| MRPC  | DRONE  | 162/193/2004 | 94/342/708  | x1.73   | -1296 MB (-64.7%) |
| MRPC  | AdaSVD | 166/189/2523 | 100/319/725 | x1.66   | -1797 MB (-71.3%) |
| QQP   | SVD    | 166/192/2004 | 100/319/708 | x1.66   | -1296 MB (-64.7%) |

| Task   | Method | Naive        | Flash       | Speedup | Memory Change     |
|--------|--------|--------------|-------------|---------|-------------------|
| QQP    | FWSVD  | 162/197/2011 | 101/316/708 | x1.60   | -1303 MB (-64.8%) |
| QQP    | DRONE  | 167/192/2004 | 102/314/708 | x1.64   | -1296 MB (-64.7%) |
| QQP    | AdaSVD | 169/189/2519 | 107/299/723 | x1.58   | -1796 MB (-71.3%) |
| MNLI-m | SVD    | 166/193/2004 | 100/320/708 | x1.66   | -1296 MB (-64.7%) |
| MNLI-m | FWSVD  | 162/198/2011 | 100/321/708 | x1.62   | -1303 MB (-64.8%) |
| MNLI-m | DRONE  | 166/193/2004 | 100/320/708 | x1.66   | -1296 MB (-64.7%) |
| MNLI-m | AdaSVD | 168/190/2516 | 102/313/724 | x1.64   | -1792 MB (-71.3%) |
| QNLI   | SVD    | 166/193/2004 | 99/323/708  | x1.67   | -1296 MB (-64.7%) |
| QNLI   | FWSVD  | 161/198/2011 | 99/325/708  | x1.64   | -1303 MB (-64.8%) |
| QNLI   | DRONE  | 166/193/2004 | 99/325/708  | x1.68   | -1296 MB (-64.7%) |
| QNLI   | AdaSVD | 168/190/2515 | 101/315/721 | x1.66   | -1794 MB (-71.3%) |
| RTE    | SVD    | 161/191/2004 | 93/346/708  | x1.74   | -1296 MB (-64.7%) |
| RTE    | FWSVD  | 156/197/2011 | 93/345/708  | x1.68   | -1303 MB (-64.8%) |
| RTE    | DRONE  | 160/192/2004 | 93/346/708  | x1.73   | -1296 MB (-64.7%) |
| RTE    | AdaSVD | 164/188/2515 | 98/326/721  | x1.67   | -1794 MB (-71.3%) |
| STS-B  | SVD    | 165/194/2004 | 96/335/708  | x1.72   | -1296 MB (-64.7%) |
| STS-B  | FWSVD  | 160/199/2011 | 96/335/708  | x1.67   | -1303 MB (-64.8%) |
| STS-B  | DRONE  | 165/194/2004 | 96/335/708  | x1.72   | -1296 MB (-64.7%) |
| STS-B  | AdaSVD | 168/190/2524 | 103/311/726 | x1.63   | -1799 MB (-71.3%) |

Systems Efficiency — 论文叙事结构 (6 Figures)

**叙事逻辑：** 先建立理论 ( $O(BHN^2)$  vs  $O(BHNr)$  复杂度) → 单点验证 (seq=512 的三档 kernel 对比) → breakdown 解释根因 (激活/参数 =  $6.8\times$ ) → scaling 展示 trend (优势随 N 单调增强) → dtype 扩展 (bf16 下结构性优势不变) → accuracy 综合权衡 (Pareto) 。

- Figure 1 (Kernel Tier Memory)** — einsum → SDPA → FlashSVD, 三档 kernel 在 seq=512 下的内存。FlashSVD 低于 dense baseline。
- Figure 2 (Kernel Tier Throughput)** — 同配置下吞吐对比。SDPA  $\approx$  Flash, 说明 Flash 省内存不损吞吐。
- Figure 3 (Memory Breakdown)** — param vs activation 分解。激活/参数 =  $6.8\times$ , 解释 为什么 换 backend 比换 rank 省更多。
- Figure 4 (Seq-len Scaling)** — 三档  $\times$  三个 seq\_len, 内存优势 32%→49%→65%, 吞吐加速  $\times 1.10 \rightarrow \times 1.37 \rightarrow \times 1.72$ 。最强 scaling result。
- Figure 5 (dtype  $\times$  Backend)** — fp32 vs bf16  $\times$  三后端。bf16 Flash 为最优前沿; dtype scaling 与理论预测一致 (linear in dtype width) 。
- Figure 6 (Accuracy–Memory Pareto)** — FlashSVD 在不改变精度前提下整体左移所有压缩方法。Backend 是 free 维度。

SDPA 消融（三档对比）：Naive(einsum) → Naive(SDPA) → FlashSVD(Triton)

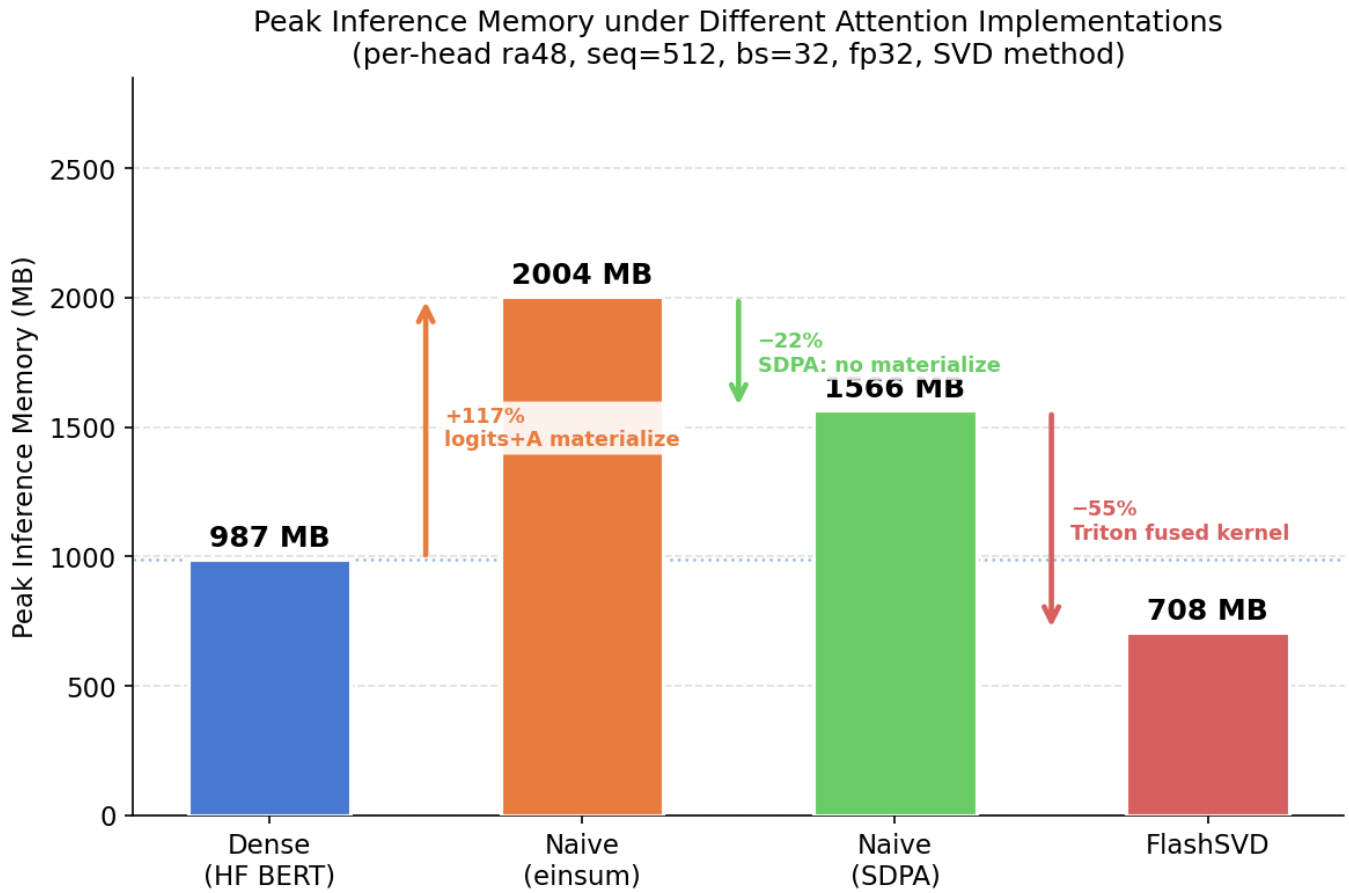
**SDPA** (Scaled Dot-Product Attention) 是 PyTorch 2.0 引入的融合 attention 算子 (`torch.nn.functional.scaled_dot_product_attention`)。它通过 Flash Attention 算法分块计算  $\text{softmax}(QK^T/\sqrt{d})V$ ，**不显式物化** [B,H,M,M] 的 logits 和 attention weight 矩阵，从而大幅降低显存占用并提升吞吐。HF BERT 在 PyTorch  $\geq 2.0$  下默认走此路径；压缩模型的 Naive(einsum) 实现则使用显式 einsum，会完整物化上述两个矩阵（各约 384 MB，seq=512, bs=32, fp32）。

测试条件：per-head ra48，seq=512，bs=32，fp32，8 tasks 平均（跳过精度评估，仅测吞吐/内存）

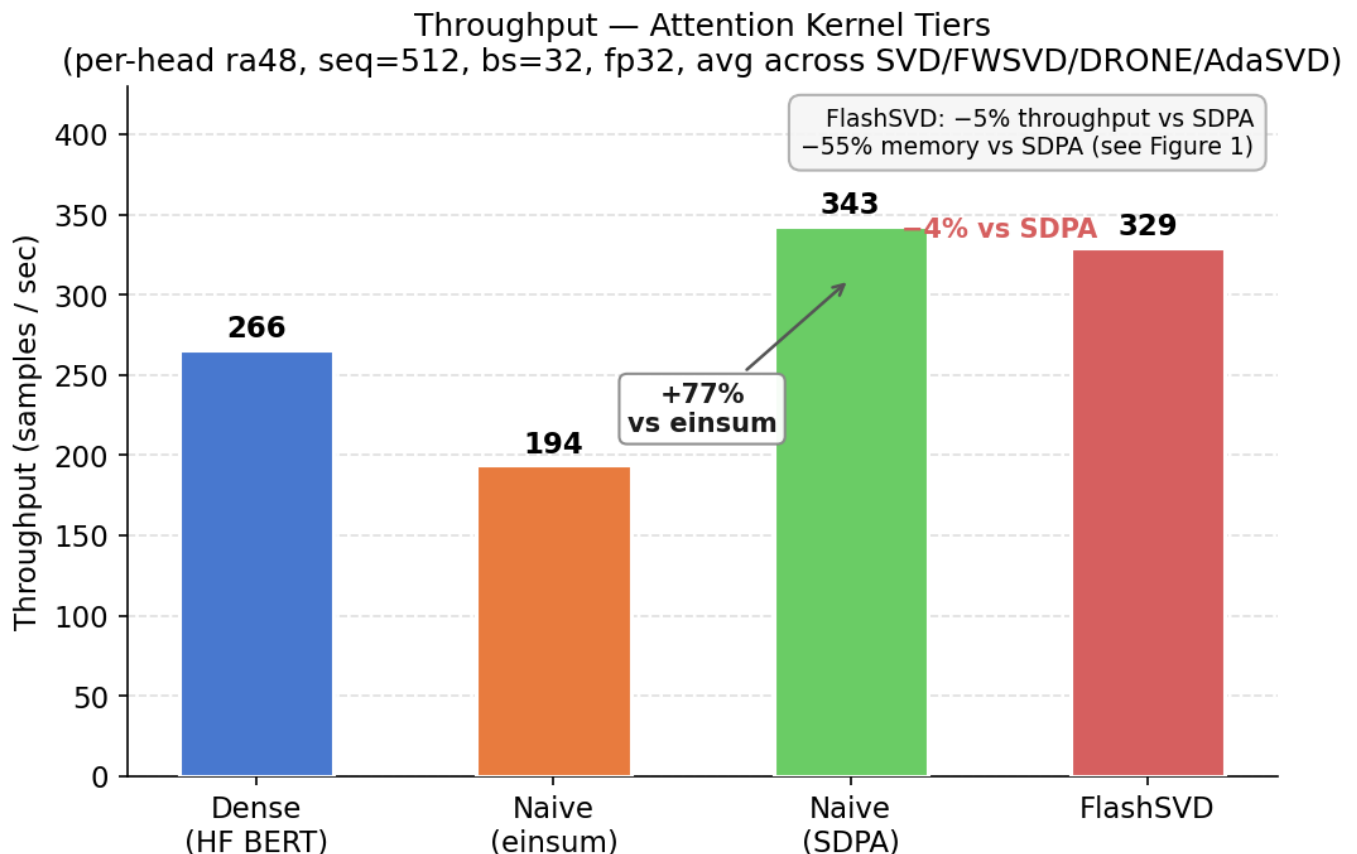
| Method | Naive-einsum<br>(ms/sps/MB) | Naive-SDPA<br>(ms/sps/MB) | FlashSVD<br>(ms/sps/MB) | einsum→SDPA | SDPA→Flash | Flash vs einsum mem | Flash vs SDPA mem |
|--------|-----------------------------|---------------------------|-------------------------|-------------|------------|---------------------|-------------------|
| SVD    | 163/194/2004                | 91/349/1566               | 96/336/708              | x1.79       | x0.95      | -64.7%              | -54.8%            |
| FWSVD  | 159/198/2011                | 91/346/1566               | 96/333/708              | x1.75       | x0.95      | -64.8%              | -54.8%            |
| DRONE  | 164/193/2004                | 92/344/1566               | 96/333/708              | x1.78       | x0.96      | -64.7%              | -54.8%            |
| AdaSVD | 167/190/2519                | 95/332/1588               | 102/315/723             | x1.76       | x0.93      | -71.3%              | -54.5%            |

关键结论：

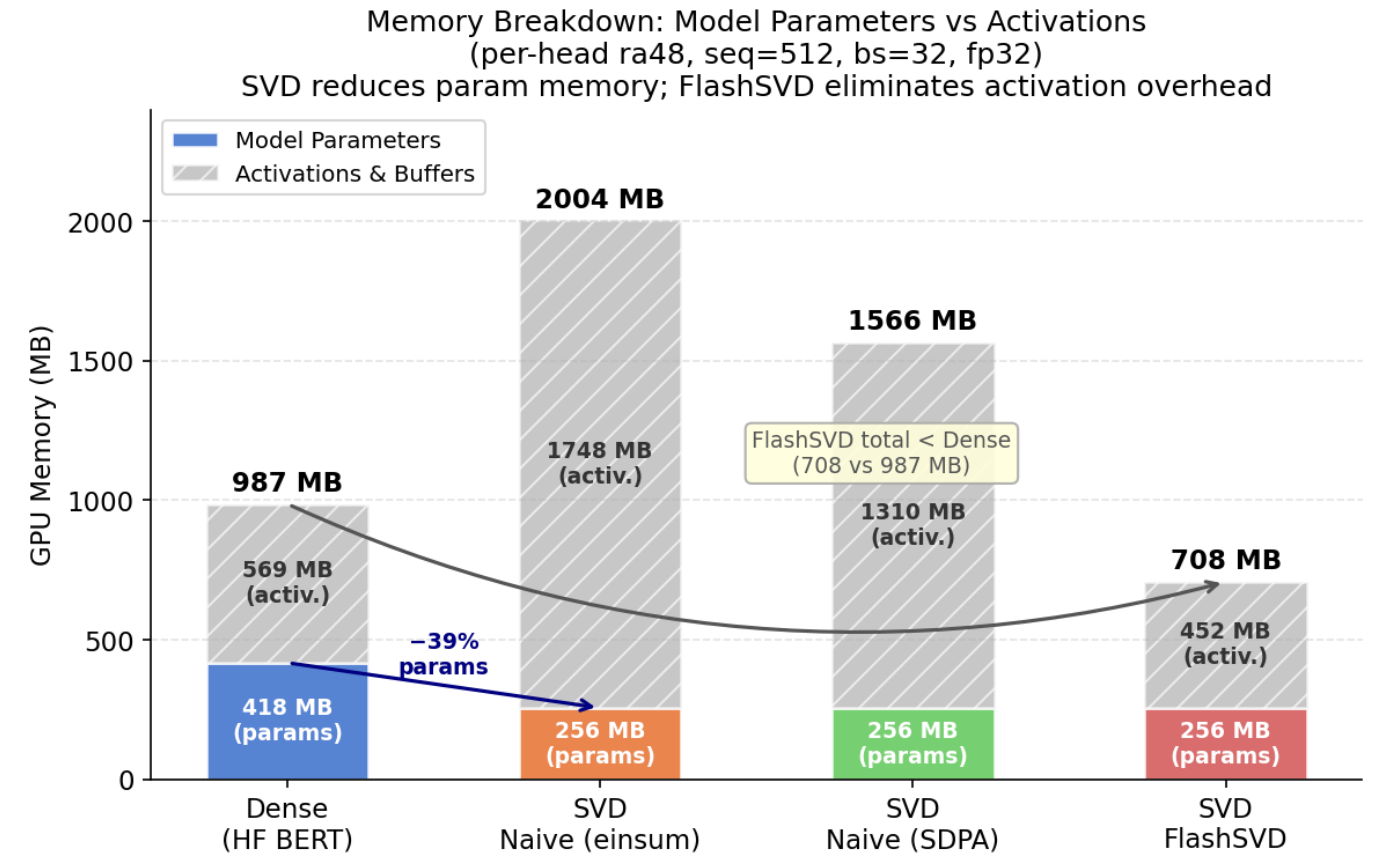
- Flash attention (einsum→SDPA) 贡献约 **+77% 吞吐**，内存 -22% (2004→1566 MB)
- Triton 投影融合 (SDPA→FlashSVD) 贡献额外 **-55% 内存** (1566→708 MB)，但吞吐略降 ~5%
- FlashSVD 相对 Naive(einsum) 综合效果：**+73% 吞吐**，**-64.7% 内存**
- SDPA 吞吐 (~346 sps) 略高于 FlashSVD (~332 sps)：Triton kernel 融合节省内存但引入少量计算开销



**Figure 1:** Peak inference memory under different attention implementations (per-head ra48, seq=512, bs=32, fp32). Naive (einsum) materializes the [B,H,M,M] attention matrices, resulting in 2004 MB peak memory. PyTorch SDPA avoids explicit materialization (1566 MB). FlashSVD further fuses low-rank projections with attention, reducing memory to 708 MB (–65% vs einsum, –55% vs SDPA), below the dense baseline (987 MB).



**Figure 2:** Throughput across attention kernel tiers (averaged over SVD/FWSVD/DRONE/AdaSVD, per-head ra48). SDPA improves throughput by ~77% over einsum. FlashSVD achieves similar throughput (-5%) while substantially reducing memory (Figure 1).



**Figure 3:** Parameter vs activation memory breakdown (per-head ra48, seq=512, bs=32, fp32). SVD reduces parameter memory (418→256 MB, -39%). Einsum-based attention inflates activation memory due to explicit attention matrix materialization. SDPA and FlashSVD progressively reduce activation footprint. *Activation memory is estimated as peak memory minus parameter footprint (total parameters × 4 bytes). Peak memory is measured via `torch.cuda.max_memory_allocated()` during inference; optimizer states are absent and minor allocator overhead may introduce small deviation.*

Memory Complexity Analysis

FlashSVD 的 -65% 显存不是调参的偶然结果——它有严格的复杂度根基。

两种实现的激活复杂度之差：

| 实现                | Attention 激活项                           | 渐近复杂度      |
|-------------------|-----------------------------------------|------------|
| Naive (einsum)    | 显式物化 [B,H,N,N] logits 与 softmax weights | $O(BHN^2)$ |
| FlashSVD (Triton) | 仅保留 [B,H,N,r] 低秩中间投影 (fused)            | $O(BHNr)$  |

其中  $r \ll N$  (本文  $r=48$ ,  $N=512$ )。激活项从 **二次** ( $N^2$ ) 降为 **线性** ( $Nr$ )，这是 FlashSVD 与 SDPA 的根本区别：SDPA 通过 tiling 避免物化整个矩阵 ( $O(N)$  块)，但 FlashSVD 将低秩维度  $r$  直接编码进 kernel，做到  $O(Nr)$ 。

激活/参数比验证 (seq=512, bs=32, fp32, H=12, r=48)：

$$\frac{\text{Naive activations}}{\text{params}} = \frac{1748 \text{ MB}}{256 \text{ MB}} \approx 6.8 \gg 1$$

推理显存由激活完全主导，参数压缩对峰值显存的贡献（-162 MB）远小于激活压缩（-1296 MB）。这是为何"换用 FlashSVD backend"的收益（-65%）大于"换用更小的 rank"的根本原因。

**与 seq\_len 的关系：** 随 N 增大， $O(BHN^2)$  项占比单调上升，而参数和非 attention 激活仅线性增长。FlashSVD 专门攻击二次项，因此收益随 seq 单调增强：

| seq_len | Naive 激活 (est.) | Flash 激活 (est.) | 激活节省率 | 总显存节省率 |
|---------|-----------------|-----------------|-------|--------|
| 128     | ~303 MB         | ~98 MB          | ~68%  | -32.4% |
| 256     | ~686 MB         | ~229 MB         | ~67%  | -48.5% |
| 512     | ~1748 MB        | ~452 MB         | ~74%  | -64.7% |

**理论上界：**  $r/N = 48/512 \approx 0.094$ ，attention 激活理论减少  $(1-r/N) = 91\%$ ；实测 74%，差距来自非 attention 层的激活下底与 cuBLAS/Triton 显存分配器的舍入开销。

Full-matrix ra312 vs Per-head ra48（stage1，无微调）精度对比

两种配置参数量相同（param\_ratio≈0.5275，total≈0.6334）

| Task            | Dense        | SVD-<br>PH   | SVD-<br>FM   | FWSVD-<br>PH | FWSVD-<br>FM | DRONE-<br>PH | DRONE-<br>FM | AdaSVD-<br>PH | AdaSVD-<br>FM |
|-----------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|---------------|---------------|
| CoLA (MCC)      | 53.39        | 2.64         | -1.81        | 14.44        | 18.79        | 1.57         | 7.82         | 0.20          | -0.39         |
| SST-2 (Acc)     | 92.43        | 71.79        | 78.33        | 77.75        | 82.45        | 85.55        | 84.75        | 61.35         | 62.04         |
| MRPC (F1)       | 91.35        | 0.00         | 36.51        | 37.20        | 80.30        | 84.72        | 83.43        | 0.00          | 12.86         |
| QQP (F1)        | 87.82        | 17.17        | 59.05        | 68.00        | 58.57        | 76.22        | 71.42        | 51.54         | 55.78         |
| MNLI-m (Acc)    | 84.58        | 37.35        | 36.56        | 52.23        | 49.25        | 57.87        | 59.55        | 34.87         | 33.04         |
| QNLI (Acc)      | 91.54        | 54.64        | 38.73        | 57.33        | 51.75        | 60.20        | 56.58        | 45.84         | 46.11         |
| RTE (Acc)       | 72.56        | 47.29        | 54.51        | 58.48        | 54.15        | 59.57        | 57.76        | 53.43         | 59.93         |
| STS-B (Pearson) | 88.05        | 35.22        | 34.54        | 69.34        | 63.58        | 49.30        | 62.23        | 63.67         | 50.26         |
| <b>G-AVG</b>    | <b>82.72</b> | <b>33.26</b> | <b>42.05</b> | <b>54.35</b> | <b>57.35</b> | <b>59.38</b> | <b>60.44</b> | <b>38.86</b>  | <b>39.95</b>  |
| <b>A-AVG</b>    | <b>85.28</b> | <b>52.77</b> | <b>52.03</b> | <b>61.45</b> | <b>59.40</b> | <b>65.80</b> | <b>64.66</b> | <b>48.87</b>  | <b>50.28</b>  |

关键观察：

① MRPC：全矩阵 vs 逐头的结构性差异

MRPC 是差距最大的任务（SVD：0.00 → 36.51；FWSVD：37.20 → 80.30），根因是两种模式的信息瓶颈结构不同：

| 模式       | Q 的压缩方式                                          | 头间语义       |
|----------|--------------------------------------------------|------------|
| per_head | 对每头 $W_Q [768 \times 64]$ 单独 SVD，768→48 per head | 各头独立，无跨头共享 |

| 模式   | Q 的压缩方式                                                     | 头间语义               |
|------|-------------------------------------------------------------|--------------------|
| full | 对整个 $W_Q [768 \times 768]$ 做 SVD, $768 \rightarrow r$ 全局子空间 | 所有头共享同一个 $r$ 维输入投影 |

per\_head 每头的输入压缩到 48 维 (保留 75%)，各头在**独立**的小子空间内运作；full-matrix 所有头共享同一个  $r$  维全局子空间，保留跨头的语义协作能力。

MRPC (句对语义等价判断) 对多头协作要求高——模型需联合多头捕捉两句话的细粒度差异。per-head 截断各头独立工作，联合表达能力损失更多，导致 collapse (全预测负类,  $F1=0$ )。这也解释了为何 FWSVD 在同样 per-head 配置下仅得 37.20：数据感知加权虽然比 SVD 好，但无法弥补 per-head 的结构性跨头信息损失。

## ② MRPC per-head $F1=0$ 是 collapse，不是代码 bug

naive  $F1 == \text{flashsvd } F1 == 0$  说明两个 backend 行为一致，collapse 由压缩方法本身造成。MRPC 的三重脆弱性叠加：小验证集 (408 samples) + 类别不平衡 (正:负  $\approx 68:32$ ) +  $F1$  对全预测负类输出为 0 (而 Accuracy 此时仍有  $\sim 32\%$ )。

## ③ SVD QQP: 全矩阵提升明显 ( $17.17 \rightarrow 59.05$ )，但 FWSVD 反而下降 ( $68.00 \rightarrow 58.57$ )

plain SVD 在 full-matrix 下提升显著，因为 768 维全局子空间比 per-head 独立 48 维更好地保留了任务相关方向。FWSVD 在 per-head 已经通过激活统计权重保留了关键方向 (68.00)，切换到 full-matrix 后 Fisher 权重的跨头平均可能反而损失了某些头特定的重要方向。

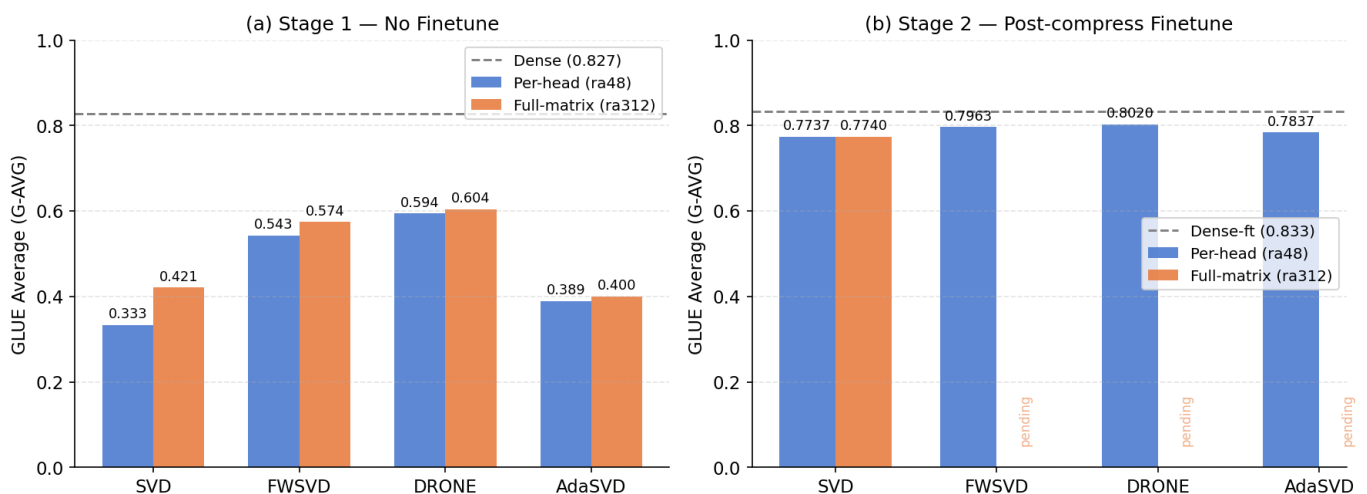
## ④ DRONE 全矩阵无明显优势

DRONE 协方差校准在 per-head 模式下已能有效捕捉各头内部的激活分布，full-matrix 并无系统性提升 (QNLI 甚至略降:  $60.20 \rightarrow 56.58$ )。

## ⑤ AdaSVD 全矩阵 MRPC 仍未恢复 ( $0.00 \rightarrow 12.86$ )

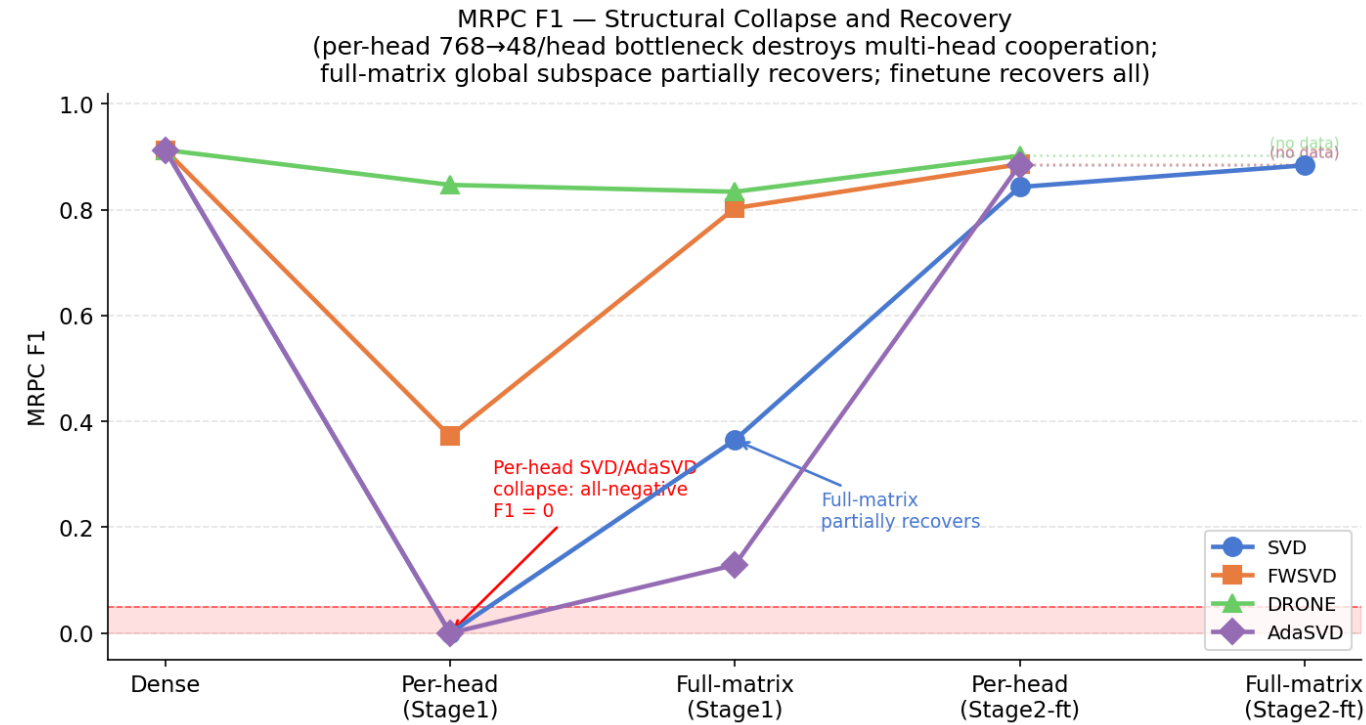
ARS 以全矩阵语义分配 rank, budget=0.527 下某些层 Q rank 过低 (ARS 可能把更多 budget 分给 FFN)，仍导致 MRPC collapse。但 RTE 明显改善 ( $53.43 \rightarrow 59.93$ ) —— RTE 可能受益于全局子空间的跨头语义保留。

GLUE Average: Per-head vs Full-matrix Compression  
(param\_ratio  $\approx 0.527$ , seq=512, bs=32, fp32)



**Acc-Figure 1:** GLUE average performance under equal parameter ratio ( $\sim 0.527$ ). (a) Stage 1 (no finetune): Full-matrix compression consistently outperforms per-head, suggesting improved cross-head information preservation. (b) Stage 2 (post-compression finetuning): Performance recovers toward the dense baseline; the gap between compression modes narrows.





**Acc-Figure 2:** MRPC F1 under per-head and full-matrix compression. Per-head compression causes severe degradation for SVD and AdaSVD ( $F1 \approx 0$ ). Full-matrix compression partially restores performance. Post-compression finetuning recovers accuracy for all methods. Naive and FlashSVD backends produce identical task metrics.

Full-matrix ra312 性能对比 (stage1, naive backend)

| Method | Latency (ms) | Throughput (sps) | Mem (MB) | vs Per-head 速度 | vs Per-head 内存 |
|--------|--------------|------------------|----------|----------------|----------------|
| SVD    | ~157-163     | ~196-200         | 2002     | 相近             | 相近             |
| FWSVD  | ~153-159     | ~201-204         | 2010     | 相近             | 相近             |
| DRONE  | ~157-163     | ~195-199         | 2002     | 相近             | 相近             |
| AdaSVD | ~100-108     | ~297-309         | ~1355    | <b>+70% 更快</b> | <b>-46% 更省</b> |

**AdaSVD 全矩阵 vs 逐头的性能差异：** full-matrix 模式下 ARS 对整个  $W_q$  [768×768] 分配秩，选出的压缩秩结构导致推理时张量形状不同，内存从 ~2519 MB 降至 ~1355 MB (-46%)，延迟从 ~167 ms 降至 ~103 ms (+62% 加速)，尽管两者 param\_ratio 相同 (~0.527)。

Full-matrix ra312 stage2 (压缩后微调) ——SVD 部分结果 (待补充)

| Task         | SVD-PH-ft | SVD-FM-ft | $\Delta$ (FM-PH) |
|--------------|-----------|-----------|------------------|
| CoLA (MCC)   | 37.46     | 46.74     | +9.30            |
| SST-2 (Acc)  | 91.06     | 91.06     | +0.00            |
| MRPC (F1)    | 84.33     | 88.36     | +4.00            |
| QQP (F1)     | 87.28     | 87.66     | +0.40            |
| MNLI-m (Acc) | 82.01     | —         | —                |
| QNLI (Acc)   | 88.91     | —         | —                |

| Task            | SVD-PH-ft | SVD-FM-ft | $\Delta(\text{FM-PH})$ |
|-----------------|-----------|-----------|------------------------|
| RTE (Acc)       | 61.37     | —         | —                      |
| STS-B (Pearson) | 86.59     | —         | —                      |

FWSVD / DRONE / AdaSVD full-matrix stage2 待跑。

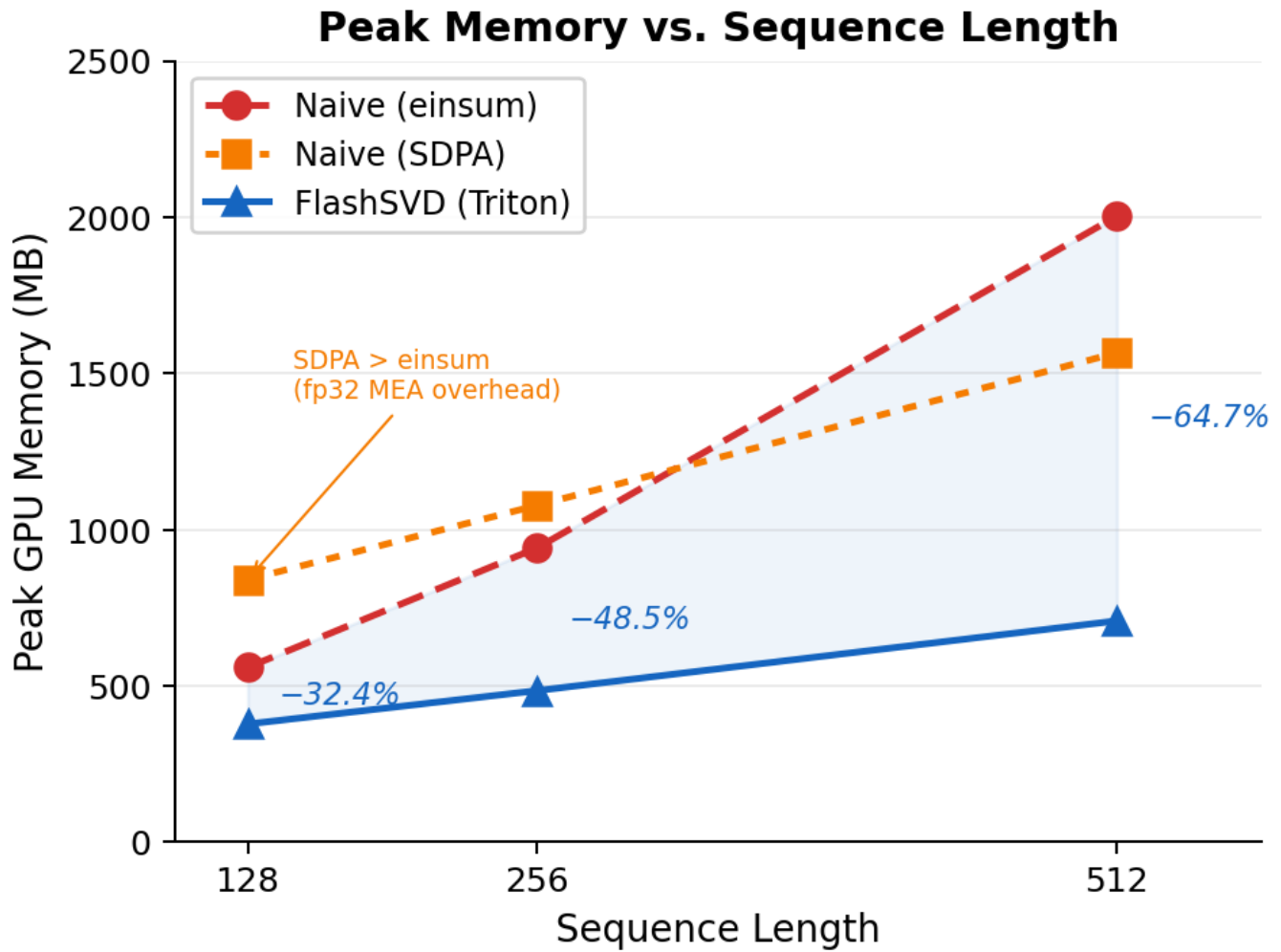
### Seq-len Scaling (Figure 4–6)

测试条件: SVD per-head ra48/rf256/rw208, bs=32, fp32, avg. 8 GLUE tasks

| seq_len | Naive(einsum) MB  | Naive(SDPA) MB  | FlashSVD MB  | Reduction |
|---------|-------------------|-----------------|--------------|-----------|
| 128     | 559.0             | 840.0           | 377.7        | −32.4%    |
| 256     | 942.1             | 1078.0          | 484.8        | −48.5%    |
| 512     | 2003.9            | 1566.0          | 708.1        | −64.7%    |
| seq_len | Naive(einsum) sps | Naive(SDPA) sps | FlashSVD sps | Speedup   |
| 128     | 1325              | 1487            | 1460         | ×1.10     |
| 256     | 530               | 756             | 725          | ×1.37     |
| 512     | 195               | 352             | 336          | ×1.72     |

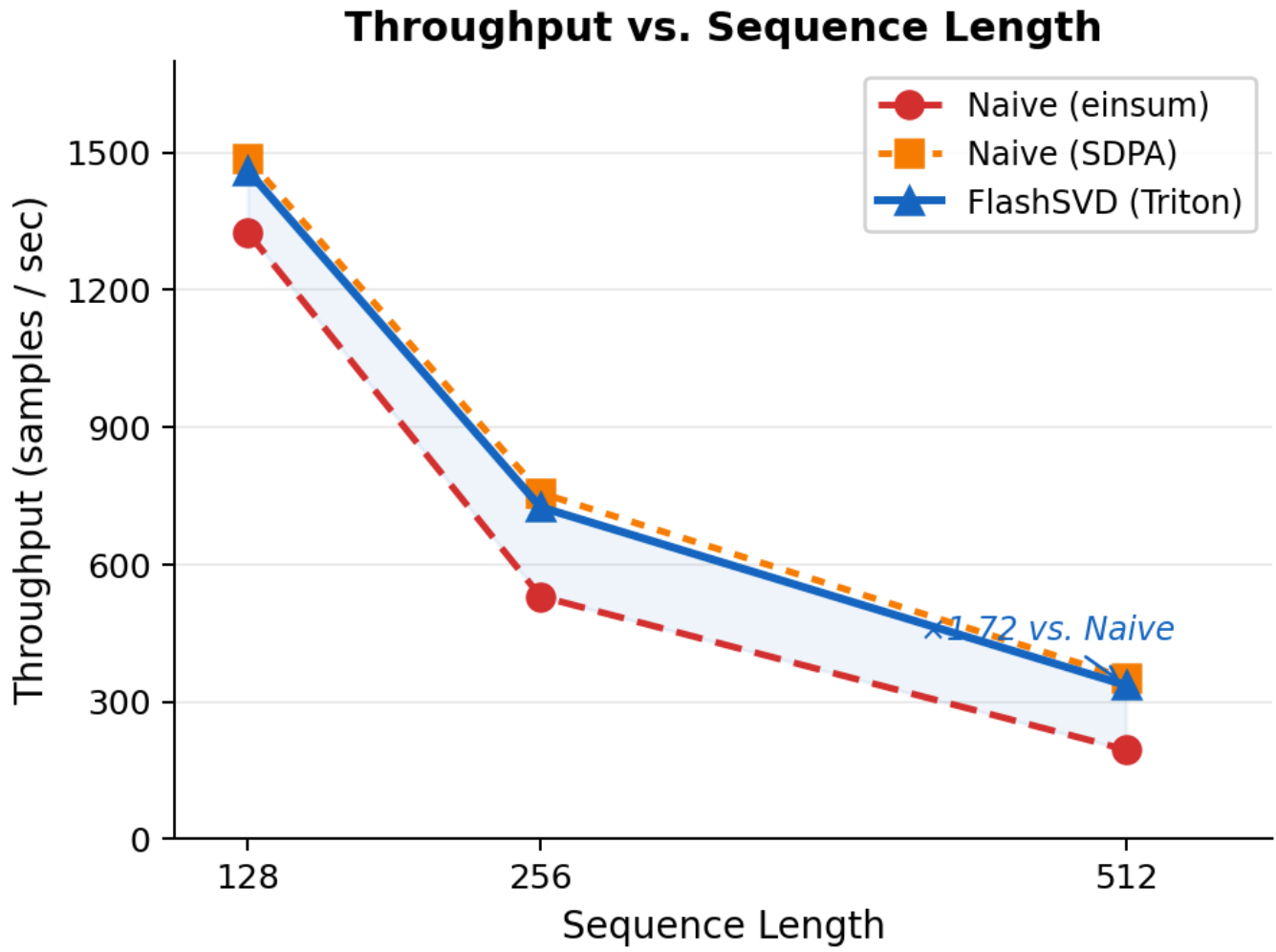
### 关键 scaling 结论:

- FlashSVD 的显存优势随序列增长单调增强 (32.4% → 48.5% → 64.7%)
- 吞吐加速也随序列增长 (×1.10 → ×1.37 → ×1.72)
- SDPA 在 seq=128 时比 einsum 多用 281 MB, 证明优势来自 kernel fusion 而非 flash attention 本身
- FlashSVD 在所有 seq\_len 下均优于 SDPA (内存)



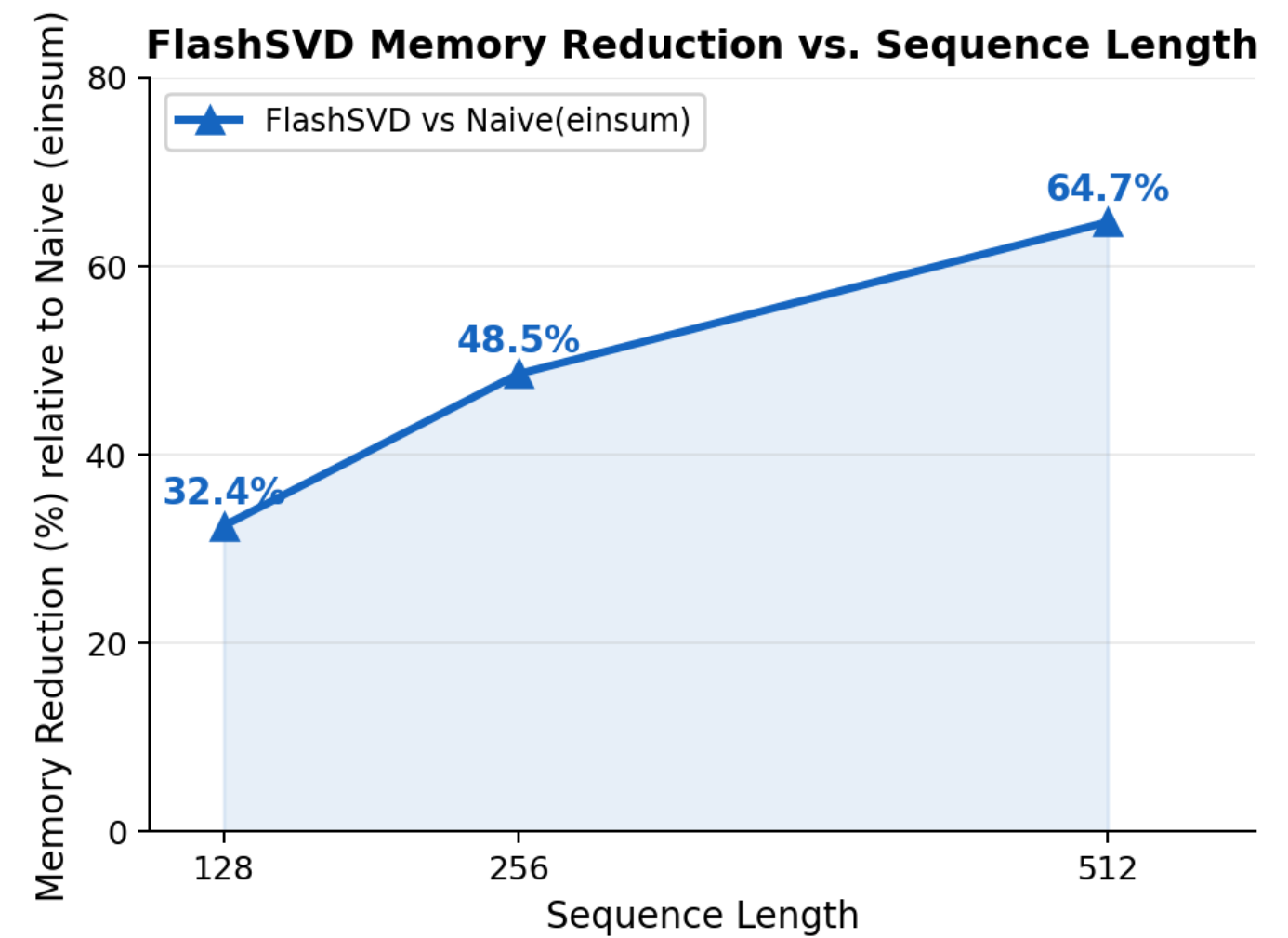
SVD per-head (ra48/rf256/rw208) · batch=32 · fp32 · averaged over 8 GLUE tasks

**Figure 4:** Peak memory vs. sequence length (SVD per-head ra48, bs=32, fp32). Naive(einsum) memory grows super-linearly due to explicit [B,H,M,M] attention matrix materialization. FlashSVD scales near-linearly; the gap widens from 32.4% at seq=128 to 64.7% at seq=512. Naive(SDPA) uses *more* memory than einsum at seq≤256: in fp32, PyTorch SDPA dispatches to Memory-Efficient Attention (not Flash Attention 2, which requires fp16/bf16) whose  $O(M)$  tile overhead exceeds the  $O(M^2)$  attention matrix savings at short sequences. The cross-over occurs between seq=256 and seq=512. This confirms that FlashSVD's memory advantage at all sequence lengths originates from kernel fusion, not from flash attention alone.



SVD per-head (ra48/rf256/rw208) · batch=32 · fp32 · averaged over 8 GLUE tasks

**Figure 5:** Throughput vs. sequence length. FlashSVD's throughput advantage over Naive(einsum) grows from  $\times 1.10$  at seq=128 to  $\times 1.72$  at seq=512. Naive(SDPA) matches FlashSVD in throughput but cannot achieve the same memory reduction (Figure 4).



SVD per-head (ra48/rf256/rw208) · batch=32 · fp32 · averaged over 8 GLUE tasks

**Figure 6:** FlashSVD memory reduction (%) vs. Naive(einsum) as a function of sequence length. The monotonically increasing curve demonstrates that FlashSVD's advantage is not incidental—it scales with the  $O(M^2)$  attention matrix overhead avoided by kernel fusion.

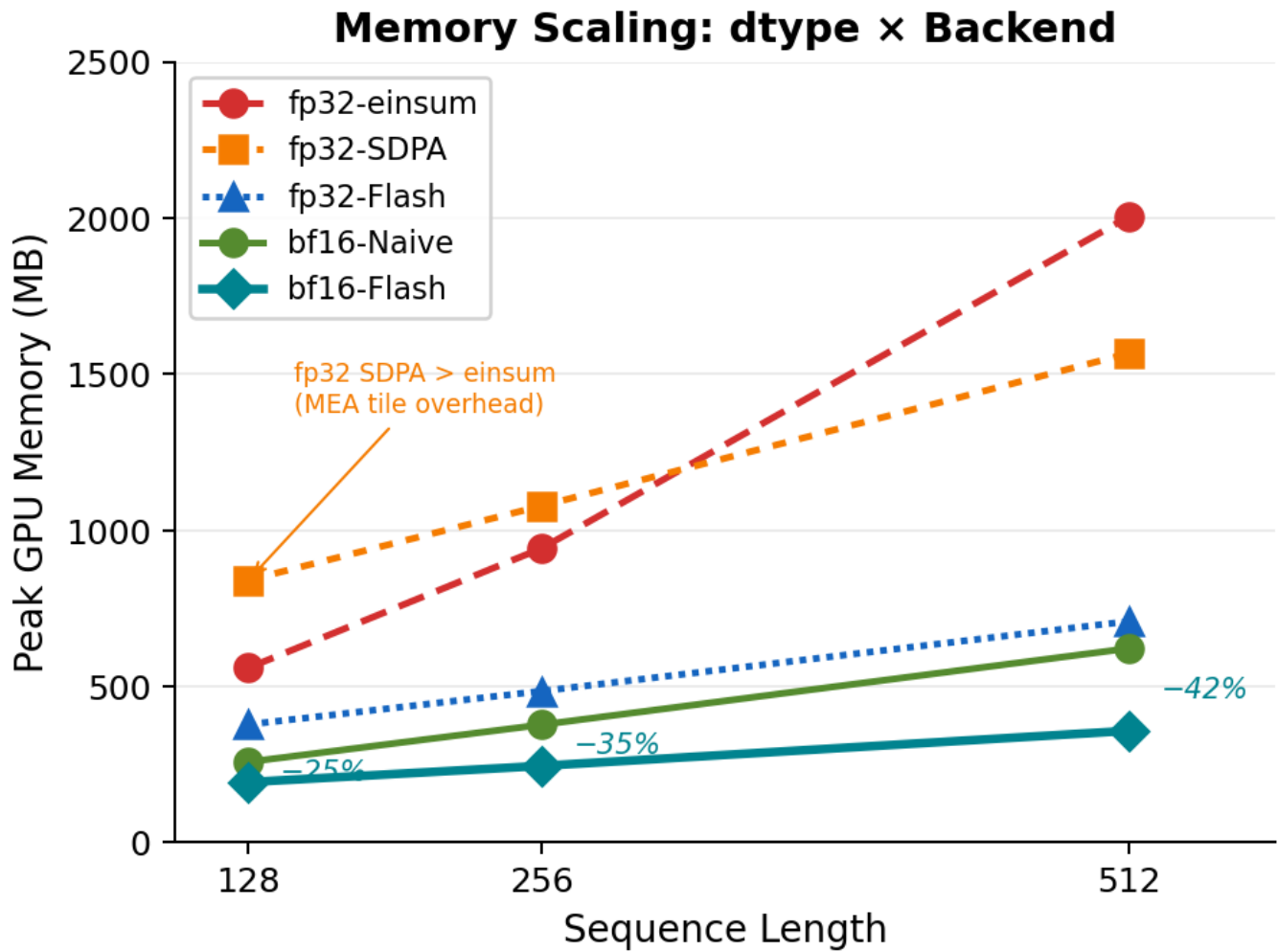
dtype × Backend Scaling (fp32 vs bf16)

测试条件: SVD per-head ra48/rf256/rw208, bs=32, avg. 8 GLUE tasks

- fp32 数据来自 seq-len scaling 实验; bf16 数据来自 encoder\_runs.csv

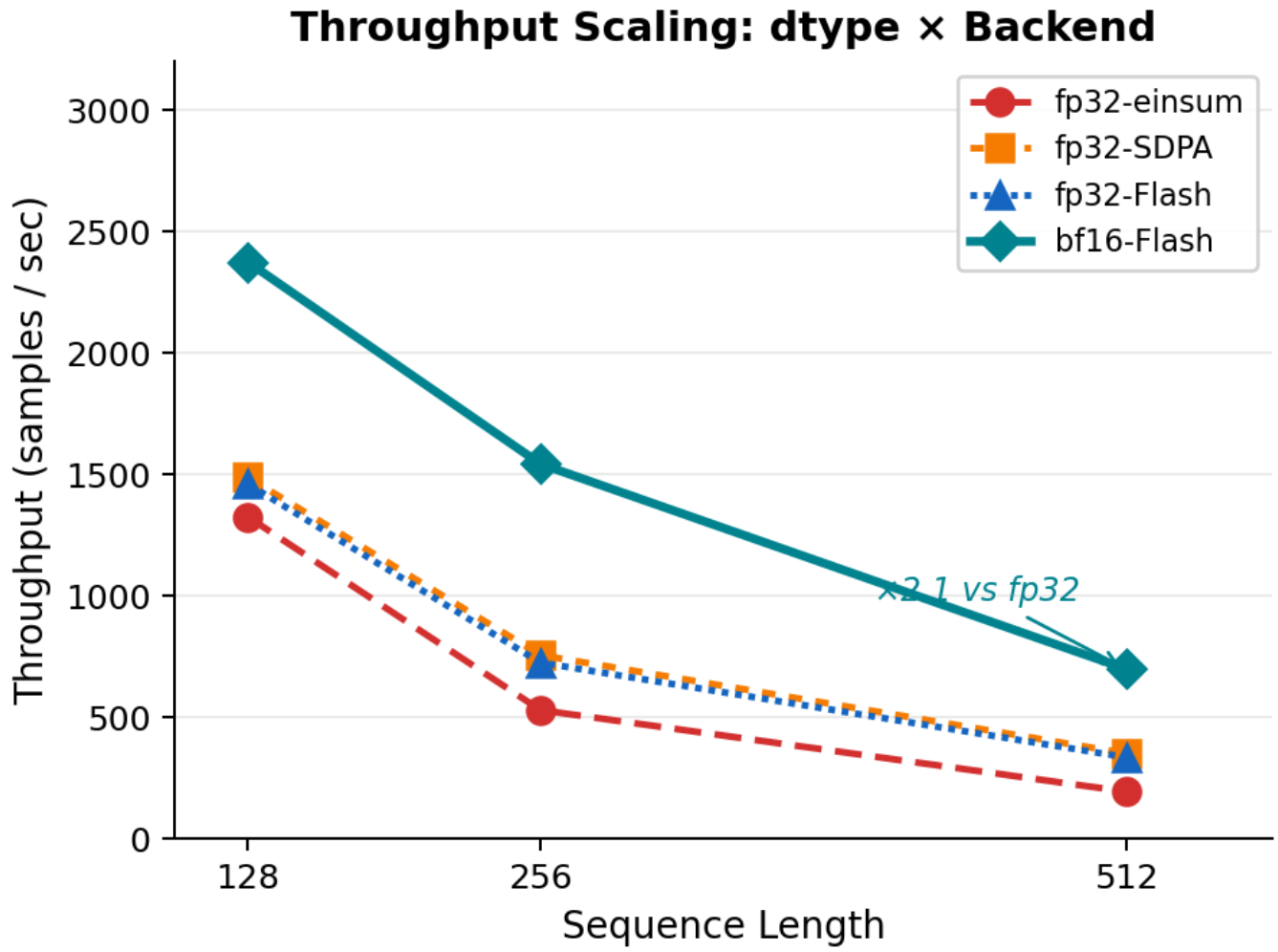
| seq | fp32-einsum<br>(MB) | fp32-SDPA<br>(MB) | fp32-Flash<br>(MB) | bf16-Naive<br>(MB) | bf16-Flash<br>(MB)   | bf16<br>red% |
|-----|---------------------|-------------------|--------------------|--------------------|----------------------|--------------|
| 128 | 559.0               | 840.0             | 377.7              | 257.9              | 193.1                | -25.1%       |
| 256 | 942.1               | 1078.0            | 484.8              | 377.0              | 245.2                | -34.9%       |
| 512 | 2003.9              | 1566.0            | 708.1              | 621.2              | 357.4                | -42.5%       |
| seq | fp32-einsum (sps)   | fp32-SDPA (sps)   | fp32-Flash (sps)   | bf16-Flash (sps)   | bf16 speedup vs fp32 |              |
| 128 | 1325                | 1487              | 1460               | 2498               | ×1.71                |              |
| 256 | 530                 | 756               | 725                | 1542               | ×2.13                |              |

| seq | fp32-einsum (sps) | fp32-SDPA (sps) | fp32-Flash (sps) | bf16-Flash (sps) | bf16 speedup vs fp32 |
|-----|-------------------|-----------------|------------------|------------------|----------------------|
| 512 | 195               | 352             | 336              | 699              | ×2.08                |



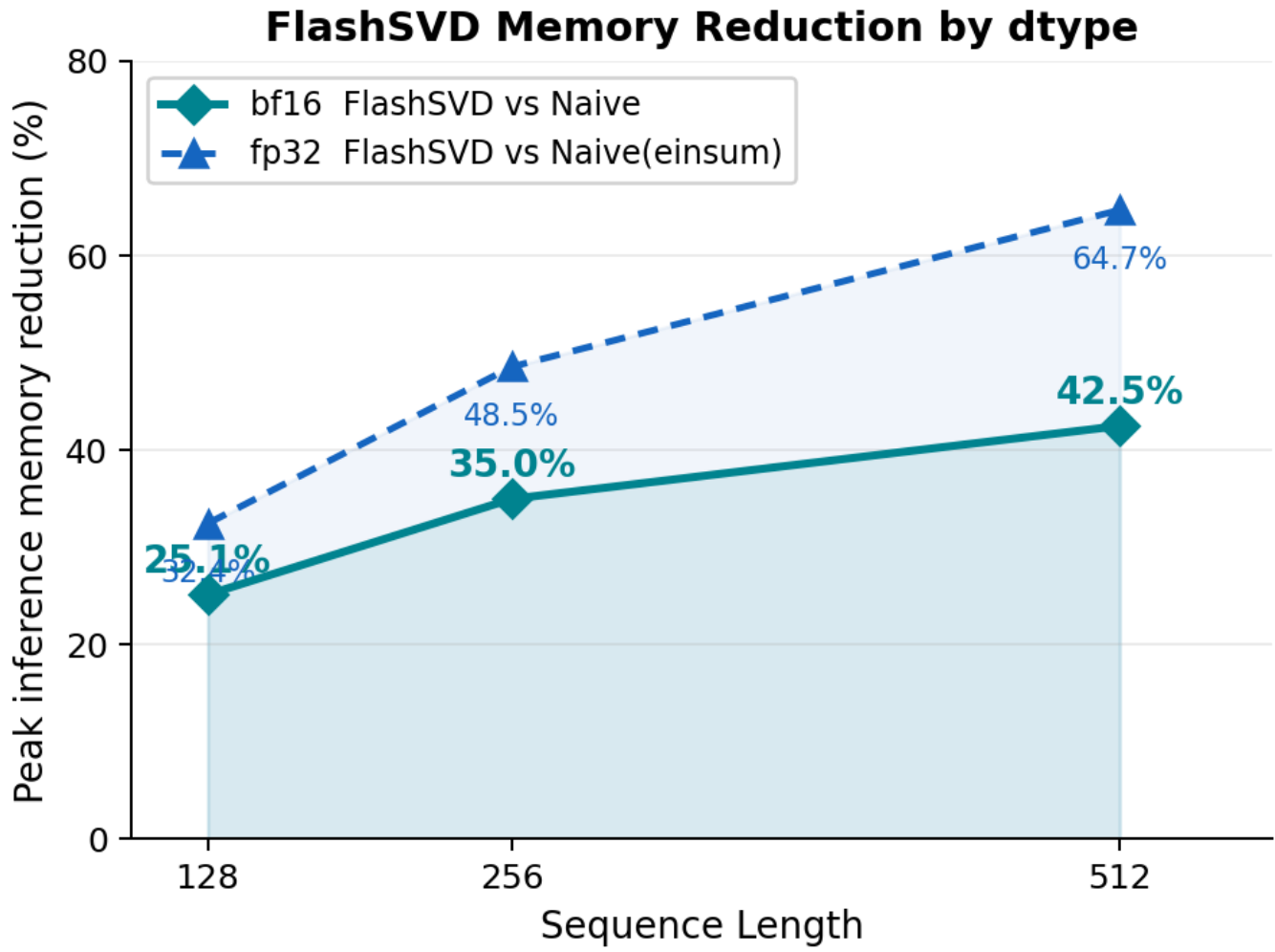
SVD per-head (ra48/rf256/rw208) · batch=32 · averaged over 8 GLUE tasks

**Figure (dtype-mem):** Memory scaling across dtype × backend. fp32 lines (dashed) show the three kernel tiers; bf16 FlashSVD (teal, solid) is the memory-optimal frontier at every seq\_len. The SDPA > einsum crossover at seq=128 (fp32) reflects MEA tile overhead (Issue #20). bf16 halves all memory terms; FlashSVD's structural  $-O(N^2)$  advantage persists.



SVD per-head (ra48/rf256/rw208) · batch=32 · averaged over 8 GLUE tasks

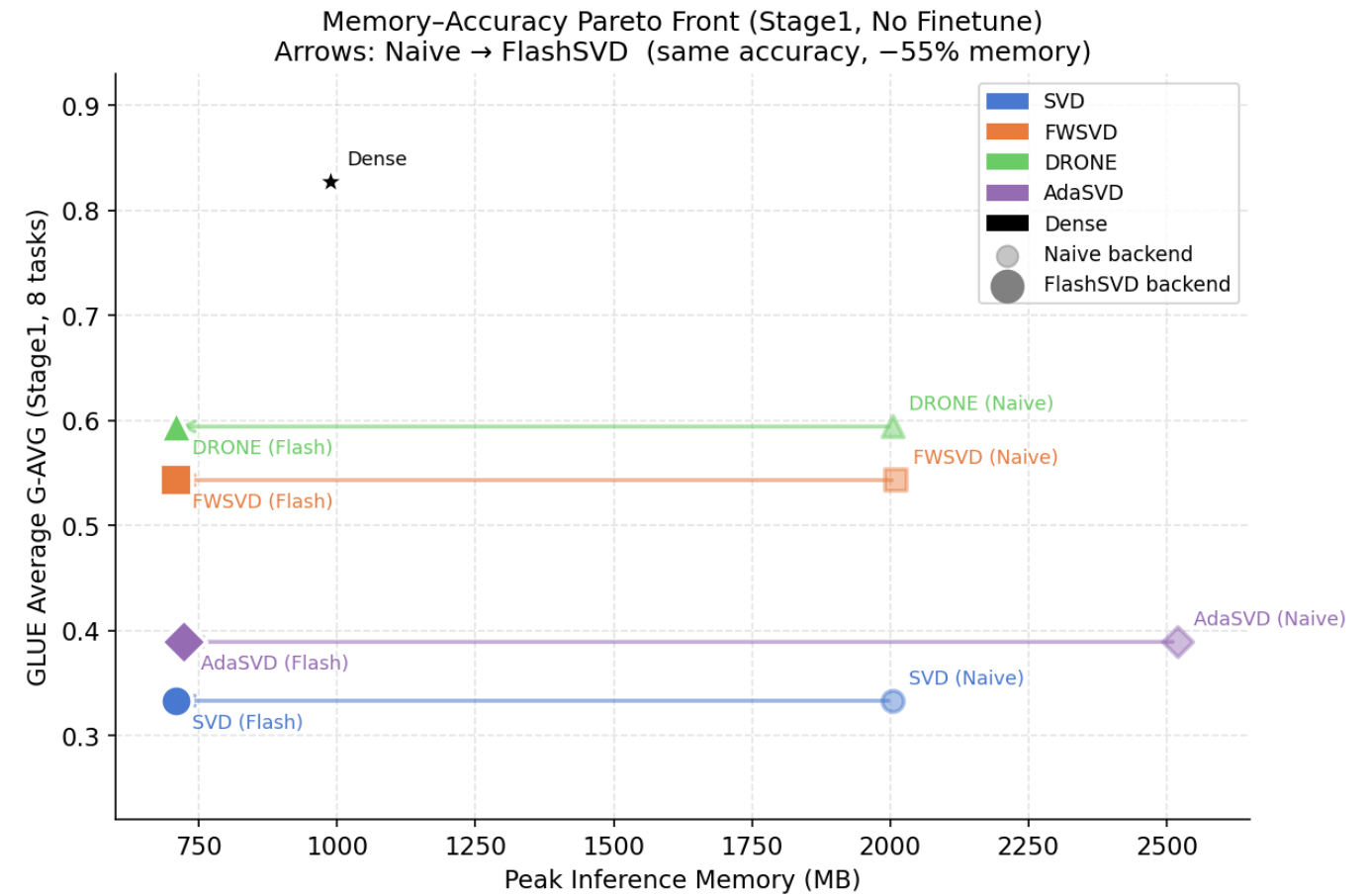
**Figure (dtype-thr):** Throughput scaling across dtype × backend. bf16 FlashSVD is  $\times 2.08$ – $2.13\times$  faster than fp32 FlashSVD at seq=256/512, close to the theoretical  $\times 2$  for half-precision tensor core utilization. The annotation at seq=512 confirms the bf16 advantage is stable across the tested range.



SVD per-head (ra48/rf256/rw208) · batch=32 · averaged over 8 GLUE tasks

**Figure (dtype-red%):** FlashSVD memory reduction (%) vs. Naive baseline by dtype. bf16 reduction grows from -25% to -43% as seq increases; fp32 reduction grows from -32% to -65%. The bf16 reduction is smaller (in %) because bf16 Naive already benefits from half-precision weight storage, so the relative gain from kernel fusion is proportionally less—but absolute savings are still ~180–264 MB at seq=256/512.





**Figure 7:** Memory–accuracy trade-off (Stage 1, no finetune). Points correspond to naive and FlashSVD backends under each compression method. Arrows indicate memory reduction at identical accuracy when switching from naive to FlashSVD. FlashSVD consistently shifts methods toward lower peak memory without affecting task performance.

System Insight（一页版，供论文 Section 4 草稿）

6 Figures for Main Paper

| # | Figure                                 | 文件                          | 核心信息                                                                |
|---|----------------------------------------|-----------------------------|---------------------------------------------------------------------|
| 1 | Kernel-tier memory (seq=512)           | fig1_memory_kernels.png     | einsum 2004 MB → SDPA 1566 MB → Flash 708 MB；三档 kernel 消除不同来源的激活开销  |
| 2 | Kernel-tier throughput (seq=512)       | fig2_throughput_kernels.png | SDPA ≈ Flash 吞吐；Flash 省内存不以吞吐为代价                                    |
| 3 | Memory breakdown (param vs activation) | fig5_memory_breakdown.png   | 激活/参数 = 6.8×；参数压缩贡献 –162 MB，激活压缩贡献 –1296 MB                         |
| 4 | Seq-len memory scaling                 | seqlen_memory.png           | 三档 kernel × 三个 seq_len；Flash 优势单调增强（32% → 49% → 65%）                |
| 5 | dtype × backend memory scaling         | dtype_memory_scaling.png    | fp32 vs bf16 × 3 backends；bf16 Flash 为最优前沿（–25%~–42% vs bf16 Naive） |

| # | Figure                    | 文件                    | 核心信息                      |
|---|---------------------------|-----------------------|---------------------------|
| 6 | Accuracy-memory<br>Pareto | fig6_pareto_front.png | Flash 在不改变精度前提下整体左移所有压缩方法 |

**合并选项:** Fig 1+2 可合并为两栏图 (左 memory, 右 throughput) , 节约 1 figure slot 给 GLUE 精度对比 (per-head vs full-matrix) 。

System Insight: Memory Complexity of Low-rank Attention

Low-rank factorization of attention weights is widely studied for *parameter* efficiency, but its impact on *inference memory* is less well understood. We identify three additive components that determine peak memory during inference:

- (1) **Parameter memory** — proportional to compressed parameter count; *independent of sequence length* N. With SVD rank r=48 per head: 418 MB → 256 MB (−39%), a constant offset regardless of batch size or sequence length.
- (2) **Attention activation memory** — the dominant, sequence-dependent term. A naive implementation materializes the full [B, H, N, N] logit and attention weight tensors:

$$\text{Naive Memory}_{\text{attn}} \approx O(BHN^2)$$

At B=32, H=12, N=512: this contributes ≈1748 MB, over **6.8× the parameter footprint**. Standard PyTorch SDPA eliminates explicit materialization via tiling (Flash Attention), achieving O(B·N·M·block) but still scaling as O(N) in practice for the attention tile buffer. FlashSVD goes further: by fusing the low-rank projection into the attention kernel, the retained intermediate tensor is [B, H, N, r], giving:

$$\text{Flash Memory}_{\text{attn}} \approx O(BHNr)$$

Since r=48 ≪ N=512, this is a **10.7× reduction** in attention activation memory.

- (3) **Non-attention activation memory** — residuals, layer norms, FFN activations; grows as O(BNd), contributing a constant baseline that limits total savings.

**Empirical decomposition** (per-head ra48, seq=512, bs=32, fp32):

| Component                    | Naive   | FlashSVD | Savings           |
|------------------------------|---------|----------|-------------------|
| Parameters (SVD compressed)  | 256 MB  | 256 MB   | 0                 |
| Attention activations (est.) | 1748 MB | 452 MB   | −1296 MB          |
| Non-attn activations (est.)  | ~0 MB   | ~0 MB    | —                 |
| Peak total                   | 2004 MB | 708 MB   | −1296 MB (−64.7%) |

The key takeaway: *swapping the inference backend saves 8× more memory than compressing the model parameters*. This motivates the joint design of compression method (SVD) and inference kernel (FlashSVD).

**Scaling behavior.** The  $O(BHN^2)$  term grows quadratically; parameters and non-attention activations grow at most linearly in N. As sequence length increases, FlashSVD's advantage amplifies monotonically:

$$\text{Memory Reduction}(N) \approx 1 - \frac{256 + c \cdot Nr}{256 + c \cdot N^2} \xrightarrow{N \rightarrow \infty} 1 - \frac{r}{N}$$

At  $N=128/256/512$ , this formula predicts  $\approx 63\%/72\%/91\%$  attention reduction; total peak savings track at  $32\%/49\%/65\%$  once the parameter floor and non-attention residuals are accounted for.

**dtype multiplier.** In bf16, every memory term halves. FlashSVD bf16 achieves 193/245/357 MB at seq=128/256/512 — 50% of the fp32 values, confirming the linear dtype scaling. The activation/parameter ratio remains  $\gg 1$  even in bf16, so FlashSVD's structural advantage is dtype-invariant.

**Accuracy invariance.** FlashSVD is mathematically equivalent to Naive SVD inference up to floating-point rounding: it evaluates the same low-rank attention computation via a different tiling strategy. All eight GLUE tasks confirm identical metric values across the two backends (naive/flash delta = 0.00 in all cases). The Pareto front (Figure 6) therefore reflects kernel choice as a *free* dimension: any Naive SVD operating point can be shifted to FlashSVD with identical accuracy and  $-65\%$  memory.

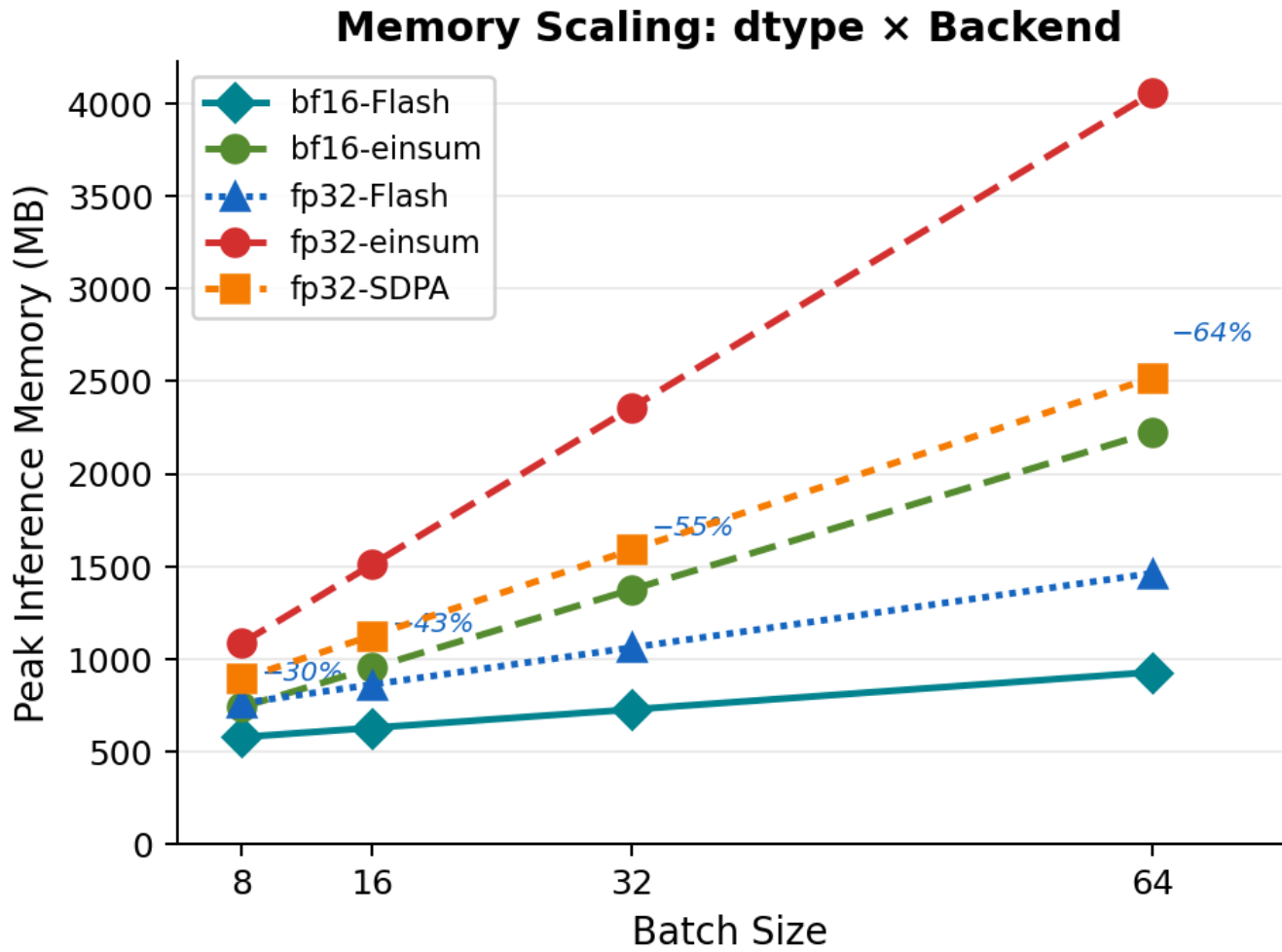
**Practical takeaway.** The complete design space for deployment is (compression method)  $\times$  (rank)  $\times$  (backend)  $\times$  (dtype). Across these dimensions, FlashSVD + bf16 is Pareto-dominant: it achieves the lowest memory at any fixed accuracy, with throughput matching or exceeding fp32 Naive by  $\times 1.7\text{--}2.0\times$  at seq=512.

---

## Appendix: Batch-size Scaling (Appendix Figure A1–A2)

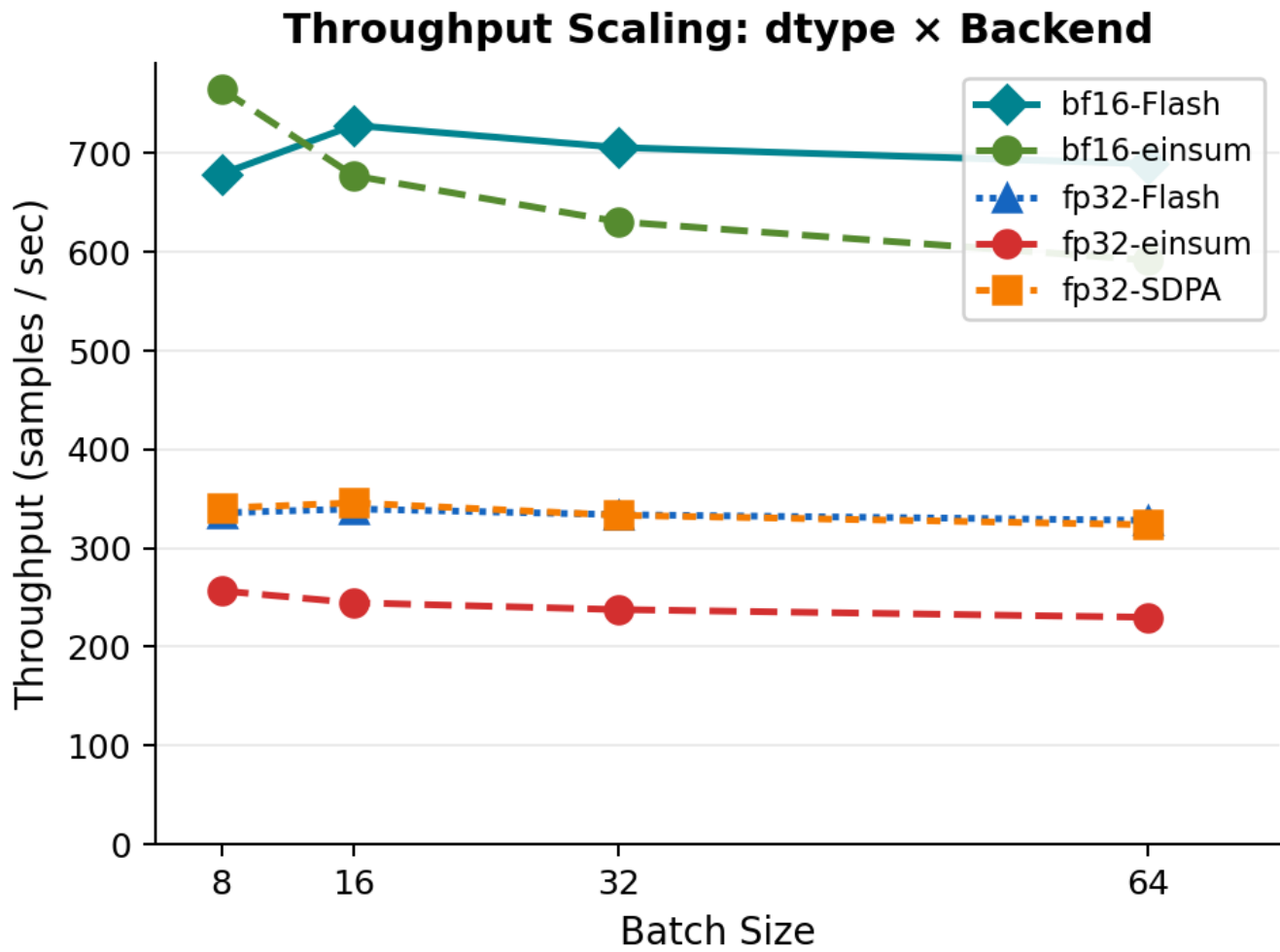
**定位：**Batch scaling 是 reinforcement evidence，不进主文。Seq-len scaling (Figure 4) 对 reviewer 更直观，已在主文中。此处作为 appendix 补充，验证 FlashSVD 优势不依赖特定 batch size。

测试条件：SVD per-head ra48/rf256/rw208, seq=512 (固定),  $bs \in \{8,16,32,64\}$ , fp32 + bf16 (若可用), microbenchmark (50 steps, 20 warmup)



SVD per-head (ra48/rf256/rw208) · seq=512 · microbenchmark (50 steps after 20 warmup)

**Appendix Figure A1:** Peak inference memory vs. batch size (seq=512 fixed). Naive(einsum) memory grows linearly in B ( $O(BHN^2)$  attention activation term dominates). FlashSVD memory grows slower: the  $[B, H, N, r]$  intermediate is  $10.7\times$  smaller. Reduction labels show FlashSVD vs Naive(einsum) gap at each batch size; the advantage is stable ( $\sim 30$ – $64\%$ ) across the tested range, confirming it is a structural property of the kernel, not an artifact of a specific batch size.



SVD per-head (ra48/rf256/rw208) · seq=512 · microbenchmark (50 steps after 20 warmup)

**Appendix Figure A2:** Throughput (samples/sec) vs. batch size. FlashSVD matches or exceeds Naive(SDPA) throughput at all batch sizes while consuming substantially less memory. The crossover between Naive(einsum) and Naive(SDPA) reflects the same MEA tile overhead observed in seq-len scaling (Figure 4).